

KARAOKÊ BORA CANTAR

Documentação Técnica Completa

Guia de Funções, Componentes e Arquitetura

Versão 1.0 | Maio de 2026

CATEGORIA	ARQUIVOS	DESCRIÇÃO
Frontend (Karaoke)	20+ arquivos	React 19 + Vite 7 + Tailwind 4 + shadcn/ui
Backend (API Server)	10+ arquivos	Express 5 + Drizzle ORM + PostgreSQL
Bibliotecas (lib/)	4 pacotes	OpenAPI, React Query hooks, Zod schemas, DB
Banco de Dados	5 tabelas	Músicas, Sessões, Usuários, Códigos de Acesso
Scripts	1 script	Importação de catálogo .ini para PostgreSQL
Deploy	Replit Workflows	Proxy reverso com path-based routing

1. VISÃO GERAL DA ARQUITETURA

O Karaokê Bora Cantar é uma plataforma profissional de karaokê construída como monorepo usando pnpm workspaces. O sistema é dividido em três camadas principais: Frontend (interface do usuário), Backend (API REST) e Banco de Dados (PostgreSQL).

1.1 Estrutura de Diretórios

O projeto segue a estrutura padrão de monorepo da Replit:

```
artifacts/ → Aplicações deployáveis karaoke/ → SPA React (frontend do usuário) api-server/ → Servidor Express (backend API) mockup-sandbox/ → Servidor de preview de componentes lib/ → Bibliotecas compartilhadas api-spec/ → Especificação OpenAPI (contratos) api-client-react/ → Hooks React Query gerados por Orval api-zod/ → Schemas Zod gerados por Orval db/ → Drizzle ORM + schemas do banco scripts/ → Scripts utilitários import-musicas.ts → Importa catálogo .ini para PostgreSQL
```

1.2 Fluxo de Dados

1. Usuário acessa a aplicação via navegador (React SPA) 2. Frontend faz requisições HTTP para /api/* (Express) 3. Backend valida inputs com Zod schemas e executa queries Drizzle 4. PostgreSQL persiste os dados (músicas, sessões, usuários) 5. Resposta JSON é consumida pelo frontend via React Query hooks

2. FRONTEND — PÁGINAS (PAGES)

As páginas são os componentes React de nível mais alto que correspondem às rotas da aplicação. Cada página é responsável por uma tela específica do sistema. Todas estão em: `artifacts/karaoke/src/pages/`

2.1 Home (home.tsx)

PÁGINA PRINCIPAL da aplicação. Exibe o catálogo de músicas (~80.000), estatísticas do catálogo, botões para iniciar sessão na TV em modo Casa ou Festa, e barra de busca persistente. Acessível na rota: `/`

Funções e Estados:

`Home()` — Componente principal da página inicial. Coordena busca, paginação, criação de sessão e diálogo de adição à fila.

```
handleStartTV(mode?: 'home' | 'party'): void → Cria nova sessão na TV ou reutiliza sessão existente → Se 'mode' diferente do atual, atualiza modo da sessão → Redireciona para /tv/{sessionId} via navigate() → Usa: createSession(), setMode(), navigate()
handleAddToQueue(singerName: string): void → Adiciona música pendente (pendingItem) à fila da sessão ativa → Se não houver sessão, cria automaticamente no modo 'home' → Mostra toast de sucesso: 'Adicionado à fila!' ou erro → Usa: createSession(), addToQueue(), toast()

Estados principais: page: number → Página atual da busca paginada startingSession: boolean → Desabilita botões durante criação pendingItem: QueueCandidate | null → Música aguardando nome do cantor scoringEnabled: boolean → Toggle pontuação (persiste no localStorage) searchTerm: string → Termo de busca (do SearchContext) debouncedSearch: string → Debounce de 300ms no searchTerm
```

Componentes filhos usados: - `Layout` (envolve toda a página) - `Button` (shadcn/ui — iniciar sessão) - `Card` + `CardContent` (grid de músicas) - `Skeleton` (estado de carregamento) - `AddToQueueDialog` (modal para nome do cantor) - `SearchProvider` (contexto de busca compartilhado)

2.2 Player (player.tsx)

PÁGINA DO PLAYER INDIVIDUAL. Exibe um vídeo específico em tela cheia com informações da música, pontuação, botão de replay e busca persistente no topo. Rota: `/player/:id`

Funções Internas:

```
randomScore(): number → Gera pontuação aleatória entre 80 e 100 → Usado para exibir pontuação após término do vídeo
PersistentSearchBar(): JSX.Element → Barra de busca fixa no topo do player → Dropdown com resultados quando focada → Debounce de 300ms na busca → Permite adicionar músicas diretamente à fila → Fecha ao clicar fora (mousedown listener)
```

Fluxo de Renderização de Vídeo:

1. Verifica se `VITE_BUNNY_LIBRARY_ID` está configurado
2. Se sim: renderiza `iframe` do Bunny Stream com `autoplay`
3. Se não: verifica vídeo local via `getFileUrl(musica.id)`
4. Se local existe: renderiza tag `<video>` com `controls`
5. Se nenhum disponível: exibe 'Carregando música...'

IMPORTANTE: O player depende da variável de ambiente `VITE_BUNNY_LIBRARY_ID`. Sem ela, o player fica travado em 'Carregando música...'. Configure no painel de Secrets do Replit.

2.3 TV Player (tv.tsx)

TELA PRINCIPAL DA TV para karaokê em grupo. Exibe fila de músicas, player de vídeo, QR Code para controle remoto, sistema de pontuação e busca integrada. É a tela mais importante do sistema. Rota: /tv/:sessionId

Funções Principais:

`generateScore(singerName: string): { score, stars, label }` → Gera pontuação aleatória entre 80 e 100 → Estrelas: 5 (95+), 4 (90+), 3 (85+), 2 (<85) → Labels: Perfeito! | Incrível! | Excelente! | Muito Bom! | Bom trabalho! → Mesma pontuação para o mesmo `singerName` (`useMemo`)

`useRouletteScore(finalScore: number): { displayed, done }` → Hook que anima contagem progressiva da pontuação → Usa `easeOutQuart` para desaceleração suave no final → Duração: 2.5 segundos → Usa `requestAnimationFrame` para animação suave a 60fps → `!displayed` = valor atual mostrado | `!done` = animação terminou

`ScoreOverlay(props): JSX.Element` → Overlay de tela cheia exibindo pontuação do cantor → Anel SVG animado com gradiente amarelo→rosa → Mostra: nome do cantor, música, artista, pontuação, estrelas, label → Próximo da fila com botão 'Começar - {nome}' → Auto-advance com countdown de 5 segundos → Responde à tecla Enter para avançar manualmente

TVPage() — Componente Principal

Gerencia todo o estado da sessão da TV: `sessionId: string` → ID da sessão (da URL, uppercase) `joined: boolean` → Se entrou na sessão com sucesso `joinError: string|null` → Erro ao entrar na sessão `showScore: boolean` → Se overlay de pontuação está visível `showSearch: boolean` → Se painel de busca está aberto `searchTerm: string` → Termo de busca no painel TV `videoKey: number` → Key para forçar re-mount do vídeo

Modos de Operação:

Modo Casa (home): → Fila ilimitada (até 30 músicas) → Qualquer dispositivo pode adicionar quantas músicas quiser → Indicado para até 4 pessoas cantando

Modo Festa (party): → Cada dispositivo só pode ter 1 música na fila por vez → Todos cantam na mesma rodada → Máximo 30 músicas na fila → Indicado para acima de 5 pessoas

Layout do Header da TV:

Esquerda (wrap livre): → Botão 'Sair' (volta para home) → Badge 'TV' → 'Sessão: L5662K' (amarelo destacado) → Botão 'Buscar' (amarelo, texto preto) → Fila de músicas (cards que quebram linha)

Direita (fixo no topo): → Badge 'Modo Casa' (verde) ou 'Modo Festa' (vermelho) → Badge 'Com Pontuação' / 'Sem Pontuação' (amarelo)

Player de Vídeo na TV: → Mesma lógica do Player individual (Bunny iframe > local > loading) → `onEnded` avança automaticamente para próxima música da fila → Se `scoringEnabled=true`, mostra `ScoreOverlay` ao terminar → Se `scoringEnabled=false`, avança direto sem pontuação

2.4 Controle Remoto (remote.tsx)

INTERFACE MOBILE para controle remoto da sessão da TV. Acessada via /remote/{sessionId}. Permite buscar músicas, adicionar à fila, gerenciar posições e trocar músicas. É a interface usada pelos convidados na festa.

Funções Principais:

RemotePage() – Componente principal → Auto-entra na sessão ao carregar (joinSession) → Duas abas: 'search' (busca) e 'queue' (gerenciar fila) → Rollup automático para sincronizar estado da sessão
handleAdd(singerName: string): void → Adiciona música pendente à fila da sessão → Valida se sessão existe e se modo permite edição → No modo Fast: verifica se dispositivo já tem música na fila
handleRemove(songId: number): void → Remove música da fila → IMPORTANTE: só quem ADICIONOU a música pode removê-la (addedBy === deviceId) → Proteção contra remoção por outros usuários
handleSwapRequest(targetIndex: number): { ok, error } → Solicita troca de posição com outro item da fila → Envia notificação ao dono do item alvo → Deve poder aceitar ou recusar
handleEdit(item: SessionQueueItem): void → Abre interface para trocar música da fila por outra → Busca nova música e substitui na fila → Mantém mesmo cantor (singerName)

Estados Principais:

tab: 'search' | 'queue' → Aba atualmente ativa pendingItem: object | null → Item aguardando nome do cantor editingItem: object | null → Item sendo editado swapSelecting: boolean → Modo de seleção para troca addError: string | null → Erro ao adicionar à fila inQueueIds: Set<number> → IDs já presentes na fila (para UI)

2.5 Painel Administrativo (admin.tsx)

DASHBOARD ADMIN para gerenciamento completo do sistema. Acessível na rota: /admin (protegido por autenticação). Contém 4 abas principais.

Abas do Painel:

CATÁLOGO: → Formulário para cadastrar nova música → Campos: ID (numérico), Artista, Música, Início (opcional) → Validação Zod: id > 0, artista/música obrigatórios → ID deve ser igual ao ID do vídeo no Bunny Stream → Sincronização automática com Bunny Stream (sync-bunny) ASSINANTES: → Lista todos os usuários (karaoke_users) → Status: Ativo | Cancelado | Expirado → Botão revogar acesso manualmente → Gera senha temporária de 8 dígitos CUPONS: → Gera códigos de acesso temporário (8 caracteres hex) → Define duração em minutos → Lista códigos ativos/usados → Acompanhamento de resgates CONFIGURAÇÕES: → Estatísticas do catálogo (total músicas, artistas) → Status da integração Bunny Stream

2.6 Login (login.tsx)

PÁGINA DE AUTENTICAÇÃO. Rota: /login. Suporta dois modos de acesso: login por email/senha (assinantes) e resgate de cupom temporário.

Modos de Login:

Login por Email/Senha: → Valida email e senha no backend → Gera token de sessão armazenado no localStorage → Redireciona para '/' se autenticado Resgate de Cupom: → Campos: Nome, Email, WhatsApp, Código do cupom → Envia para POST /api/access-codes/{code}/redeem → Salva dados no localStorage (karaoke-ct-temp-access) → Redireciona para home em caso de sucesso

2.7 Planos (planos.tsx)

PÁGINA DE PLANOS DE ASSINATURA. Rota: /planos. Exibe 3 planos com preços e recursos. Array estático PLANOS[].

Planos Disponíveis:

Mensal: R\$ 29,90/mês → Catálogo completo (~80.000 músicas) → Player Bunny Stream em HD → Fila ilimitada → 1 dispositivo por vez Trimestral: R\$ 79,90/3 meses (destaque) → Economia de R\$ 9,80 → Prioridade na fila → Suporte prioritário Anual: R\$ 249,90/ano → Economia de R\$ 108,90 → 2 dispositivos simultâneos → Conta familiar (até 4 pessoas) → Suporte VIP 24h

3. CONTEXTOS REACT (REACT CONTEXT)

Os contextos são providers React que compartilham estado global entre componentes. Cada contexto isola uma responsabilidade do sistema. Todos estão em: `artifacts/karaoke/src/contexts/`

3.1 SessionContext (session-context.tsx)

GERENCIA TODAS AS OPERAÇÕES DE SESSÃO. É o contexto mais importante do sistema, responsável por criar, entrar, atualizar e sincronizar sessões de karaokê.

Interfaces Principais:

`SessionQueueItem`: `id: number` → ID da música no catálogo `musica: string` → Título da música `artista: string` → Nome do artista `singerName: string` → Nome do cantor `addedBy: string` → ID do dispositivo que adicionou
`addedAt: string` → Data/ISO de quando foi adicionada `Session`: `id: string` → ID alfanumérico de 6 caracteres (ex: L5662K) `name: string` → Nome descritivo da sessão `mode: 'home'|'party'` → Modo de operação `queue: SessionQueueItem[]` → Array da fila de músicas `currentSongId: string|null` → ID da música atualmente tocando
`currentSingerName: string|null` → Nome do cantor atual `currentSongAddedBy: string|null` → Quem adicionou a música atual `currentSongStartedAt: string|null` → Quando a música começou `swapRequest: SwapRequestData|null` → Solicitação de troca pendente `updatedAt: string` → Última atualização da sessão

Funções do Contexto:

`createSession(name?, mode?): Promise<string|null>` → Cria nova sessão no backend via POST `/api/sessions` → Gera ID único de 6 caracteres no servidor → Salva ID no `localStorage` (`karaoke-ct-session-id`) → Inicia polling automático de 2 em 2 segundos → Retorna: ID da sessão ou null em caso de erro
`joinSession(id: string): Promise<boolean>` → Entra em sessão existente via GET `/api/sessions/{id}` → Envia `X-Device-Id` no header para identificar dispositivo → Atualiza estado local com dados da sessão → Retorna: true se sucesso, false se sessão não encontrada
`addToQueue(songId, musica, artista, singerName): Promise<{ok,error}>` → Adiciona música à fila da sessão atual → POST `/api/sessions/{id}/queue` com `X-Device-Id` → No modo Festa: backend valida se dispositivo já tem música na fila → Retorna: {ok:true} ou {ok:false, error:string}
`removeFromQueue(songId): Promise<boolean>` → Remove música da fila → DELETE `/api/sessions/{id}/queue/{songId}` → IMPORTANTE: só remove se quem solicitou for quem ADICIONOU (`addedBy === deviceId`) → Backend valida permissão: frontend mostra botão de like apenas para o dono
`playSong(songId): Promise<boolean>` → Define música como 'tocando agora' na sessão → PATCH `/api/sessions/{id}/play` com `songId` → Atualiza `currentSongId`, `currentSingerName` na sessão → Reset do vídeo (incrementa `videoKey` para re-mount)
`advanceQueue(): Promise<{ok,error}>` → Avança para próxima música da fila → Remove música atual da fila e define próxima como atual → PATCH `/api/sessions/{id}/advance`
`setMode(mode): Promise<boolean>` → Altera modo da sessão (`home` ↔ `party`) → PATCH `/api/sessions/{id}` com novo modo → Retorna: true se sucesso
`requestSwap(targetIndex): Promise<{ok,error}>` → Solicita troca de posição na fila → POST `/api/sessions/{id}/swap-request` → Envia notificação ao dono do item alvo `acceptSwap()` / `declineSwap()` → Aceita ou recusa solicitação de troca → PATCH `/api/sessions/{id}/swap-accept` ou `/swap-decline`

Polling Automático:

O contexto implementa polling automático a cada 2 segundos via `useEffect`: → `fetchSession(sessionId)` atualiza estado local → Erros são silenciosamente ignorados (`try/catch`) → Limpeza do intervalo ao desmontar componente → Sincroniza estado entre TV e controles remotos em tempo real

3.2 AuthContext (auth-context.tsx)

GERENCIA AUTENTICAÇÃO DE USUÁRIOS ASSINANTES. Suporta login por email/senha e validação de token.

Interface `AuthUser`: `id: number` → ID do usuário no banco `email: string` → Email cadastrado `subscriptionStatus: string` → Status da assinatura (`active/cancelled/expired`) `accessGranted: boolean` → Se tem acesso liberado
`accessGrantedAt: string|null` → Quando o acesso foi liberado `expiresAt: string|null` → Quando o acesso expira
Funções: `login(email, password): Promise<boolean>` → POST `/api/users/login` → Valida credenciais no backend → Armazena token no `localStorage` (`karaoke-ct-auth-token`) → Retorna true se login bem-sucedido `logout(): void` → Remove token do `localStorage` → Limpa estado do usuário → Força redirecionamento para login
`validateToken(token): Promise<AuthUser|null>` → GET `/api/users/me` com `X-Auth-Token` → Valida token no backend → Retorna dados do usuário ou null

3.3 LocalMusicContext (local-music-context.tsx)

GERENCIA VÍDEOS LOCAIS armazenados no navegador via File API. Permite selecionar uma pasta com arquivos .mp4 numerados e reproduzi-los sem Bunny Stream.

Funções: `selectFolder(): void` → Abre diálogo de seleção de pasta (`<input webkitdirectory>`) → Processa arquivos .mp4 numerados (ex: 20704.mp4) → Cria blob URLs via `URL.createObjectURL()` → Sincroniza IDs disponíveis com o backend `getFileUrl(id: number): string|null` → Retorna blob URL do vídeo local pelo ID → Retorna null se ID não existe na pasta selecionada `clearFolder(): void` → Limpa todas as URLs de blob → Revoga URLs para liberar memória → Notifica backend que não há vídeos locais

Fluxo de Uso: 1. Usuário clica 'Selecionar Pasta' na home 2. Navegador abre picker de diretório 3. Arquivos .mp4 são filtrados (apenas numéricos) 4. IDs são extraídos dos nomes de arquivo 5. `POST /api/video/sync` envia IDs para backend 6. Player usa `getFileUrl(id)` para obter vídeo local

3.4 TemporaryAccessContext (temporary-access-context.tsx)

GERENCIA ACESSO TEMPORÁRIO via cupons/códigos. Permite usar o sistema por um período limitado sem assinatura.

Interface `TemporaryAccess: code: string` → Código do cupom resgatado `durationMinutes: number` → Duração total em minutos `accessExpiresAt: string` → Data/hora de expiração (ISO) `redeemedAt: string` → Data/hora do resgate (ISO)

Funções: `redeemCode(code, name, email, whatsapp): Promise<{success,message,error}>` → `POST /api/access-codes/{code}/redeem` → Envia dados pessoais para validação → Se sucesso: salva no `localStorage` e inicia `countdown` → Se erro: retorna mensagem de erro `clearAccess(): void` → Remove acesso do `localStorage` → Zera contador de minutos restantes `hasAccess: boolean (computed)` → true se há acesso válido e não expirado → Verifica a cada 30 segundos → Redireciona para `/planos` quando expira

`Countdown Automático:` → Atualiza `remainingMinutes` a cada 30 segundos → Quando chega a 0: remove acesso e redireciona → Header mostra badge amarelo/pulso quando `<= 10` minutos

3.5 SearchContext (search-context.tsx)

GERENCIA ESTADO GLOBAL DA BUSCA. Compartilha o termo de busca entre o Layout (header) e a Home (conteúdo).

Interface: `searchTerm: string` → Termo atual de busca `setSearchTerm: function` → Atualiza termo de busca
Uso: → `Layout.tsx` lê `searchTerm` e renderiza no `input` → `Home.tsx` lê `searchTerm` via `useSearch()` e filtra resultados → `Debounce` de 300ms aplicado no hook `useDebounce`

4. HOOKS CUSTOMIZADOS (CUSTOM REACT HOOKS)

Hooks reutilizáveis que encapsulam lógica específica. Estão em: `artifacts/karaoke/src/hooks/`

4.1 useSession (use-session.ts)

Hook wrapper para acessar `SessionContext`. Lança erro se usado fora do provider.

```
function useSession(): SessionContextType → Retorna todo o contexto de sessão → Erro se não estiver dentro de <SessionProvider> → Usado em: Home, TV, Player, Remote, AddToQueueDialog
```

4.2 useScoringEnabled (use-scoring.ts)

Hook para toggle de pontuação com persistência no `localStorage`.

```
function useScoringEnabled(): [boolean, (v:boolean)=>void] → Lê estado inicial do localStorage (karaoke-ct-scoring-enabled) → Se não existir, default é TRUE (pontuação ativada) → Setter persiste no localStorage automaticamente → Usado em: Home (toggle no header), TV (condicional ScoreOverlay)
```

4.3 useDebounce (use-debounce.ts)

Hook genérico para debounce de qualquer valor.

```
function useDebounce<T>(value: T, delay: number): T → Retorna valor após 'delay' ms de inatividade → Cancela timer anterior via clearTimeout → Usado em: Home (search), Player (PersistentSearchBar), TV (search panel), Remote (search) → Delay padrão: 300ms
```

4.4 useToast (use-toast.ts)

Hook para exibir notificações toast (mensagens temporárias na tela).

```
function useToast(): { toast: function } → toast({ title, description?, variant? }) → Variantes: 'default' (info) | 'destructive' (erro) → Duração automática: ~3 segundos → Usado em: todas as páginas para feedback de ações
```

4.5 useAuth (do auth-context.tsx)

Hook wrapper para `AuthContext`. Similar ao `useSession`.

```
function useAuth(): AuthContextType → Retorna: { user, loading, error, login, logout, isAuthenticated } → Erro se usado fora de <AuthProvider> → isAuthenticated = !!user (usuário logado)
```

4.6 useTemporaryAccess (do temporary-access-context.tsx)

Hook wrapper para `TemporaryAccessContext`.

```
function useTemporaryAccess(): TemporaryAccessContextType → Retorna: { access, hasAccess, remainingMinutes, redeemCode, clearAccess } → hasAccess indica se acesso temporário está válido → remainingMinutes mostra tempo restante
```


5. BACKEND — API SERVER (EXPRESS 5)

O backend é um servidor Express 5 com rotas RESTful. Todas as rotas estão prefixadas com /api. Local: artifacts/api-server/src/

5.1 App Principal (app.ts)

Configuração central do servidor Express.

Middleware configurados: → pinoHttp: Logger HTTP estruturado (logs de requisições) → cors(): Permite requisições cross-origin → express.json(): Parse de JSON no body → express.urlencoded(): Parse de form data → router: Todas as rotas sob /api/*

5.2 Rotas de Sessões (routes/sessions.ts)

Gerencia criação, leitura, atualização e exclusão de sessões de karaokê.

Endpoints:

POST /api/sessions → Cria nova sessão → Body: { name?: string, mode?: 'home'|'party' } → Gera ID aleatório de 6 caracteres (exclui I, O, 0, 1 para evitar confusão) → Tenta até 10 vezes em caso de colisão de ID → Insere no banco: sessionsTable → Retorna: { id, name, mode } PATCH /api/sessions/:id → Atualiza modo da sessão (home ↔ party) → Body: { mode: 'home'|'party' } → Valida ID (6 caracteres) GET /api/sessions/:id → Retorna dados completos da sessão → Inclui: queue, currentSongId, currentSingerName, etc. → Usado pelo polling do frontend POST /api/sessions/:id/queue → Adiciona música à fila → Body: { id, musica, artista, singerName } → Validações: fila < 30 músicas, não duplicada → Modo Festa: verifica se dispositivo já tem música na fila → X-Device-Id identifica quem adicionou (addedBy) → Retorna: { queue: updatedQueue } DELETE /api/sessions/:id/queue/:songId → Remove música da fila → PERMISSÃO: só quem ADICIONOU pode remover (addedBy === X-Device-Id) → Se não for o dono: 403 Forbidden PATCH /api/sessions/:id/play → Define música atual como 'tocando' → Body: { songId: number } → Atualiza currentSongId, currentSingerName, currentSongAddedBy PATCH /api/sessions/:id/advance → Avança fila: remove atual, próxima vira atual → Se fila vazia: zera currentSongId POST /api/sessions/:id/swap-request → Solicita troca de posição → Body: { targetIndex: number } → Cria SwapRequestData pendente PATCH /api/sessions/:id/swap-accept / swap-decline → Aceita ou recusa solicitação de troca → Se aceitar: inverte posições dos itens na fila

5.3 Rotas de Músicas (routes/musicas.ts)

Gerencia o catálogo de músicas e busca.

Endpoints:

GET /api/musicas/stats → Retorna estatísticas do catálogo → { totalSongs, totalArtists } (apenas com vídeo disponível) GET /api/musicas/search → Busca músicas com paginação → Query params: q (busca), page, limit (default 24) → Se 'q' for número puro: busca por ID exato → Se 'q' for texto: busca I LIKE em musica, artista, inicio → Filtro: apenas músicas com hasVideo=true → Offset: (page-1) * limit GET /api/musicas/:id → Retorna música específica pelo ID → Usado para carregar info da música no player POST /api/musicas → Cadastra nova música no catálogo → Body: { id, artista, musica, inicio? } → Validação via Zod schema → ID deve ser único (conflito retorna erro) PATCH /api/musicas/:id → Atualiza dados de uma música → Body: campos a atualizar DELETE /api/musicas/:id → Remove música do catálogo POST /api/musicas/sync-bunny → Sincroniza com Bunny Stream API → Requer: BUNNY_API_KEY e BUNNY_LIBRARY_ID no servidor → Busca todos os vídeos na library → Marca músicas do catálogo como hasVideo=true quando encontradas → Retorna: { synced, totalVideos, matched, unmatched }

5.4 Rotas de Usuários (routes/users.ts)

Gerencia assinantes, autenticação e acesso.

Endpoints:

GET /api/users → Lista todos os assinantes → Ordena por ID → Oculta dados sensíveis (passwordHash, tokens) POST /api/users/login → Autenticação por email/senha → Valida email e senha com bcryptjs → Gera token aleatório (32 bytes hex) → Retorna: { token, user } GET /api/users/me → Retorna dados do usuário autenticado → Requer header: X-Auth-Token POST /api/users/:id/password → Gera senha temporária (8 dígitos numéricos) → Hash com bcryptjs → Envia por email (placeholder para futura implementação) POST /api/users/:id/revoke →

Revoga acesso do usuário → Zera accessGranted, accessGrantedAt, expiresAt

5.5 Rotas de Webhook (routes/webhook.ts)

Recebe eventos do WooCommerce para automação de assinaturas.

Endpoint: POST /api/webhook/woocommerce → Body: { event, customer_id, customer_email, subscription_status, order_id } → Eventos suportados: subscription_active → Libera acesso ao usuário order_completed → Libera acesso ao usuário subscription_cancelled → Revoga acesso subscription_expired → Revoga acesso → Se email não existir: cria novo usuário automaticamente → Se existir: atualiza status e datas → Gera senha temporária ao criar novo usuário

5.6 Rotas de Códigos de Acesso (routes/access-codes.ts)

Gerencia cupons de acesso temporário.

Endpoints:

POST /api/access-codes → Gera novo(s) código(s) de acesso → Body: { durationMinutes, quantity?, label?, createdBy? } → Gera código de 8 caracteres hex (ex: A3F9B2E1) → Verifica unicidade no banco → Retorna: { codes: [{code, durationMinutes, label, expiresAt}] } GET /api/access-codes → Lista todos os códigos → Inclui status: ativo, resgatado, expirado POST /api/access-codes/:code/redeem → Resgata cupom de acesso → Body: { name, email, whatsapp } → Valida se código existe e não foi resgatado → Calcula data de expiração: now + durationMinutes → Retorna: { durationMinutes, accessExpiresAt, message }

5.7 Rotas de Vídeo (routes/video.ts)

Serve vídeos locais do servidor.

Endpoint: GET /api/video/:id → Requer: LOCAL_MUSIC_PATH no ambiente do servidor → Sanitiza ID (apenas números, previne path traversal) → Resolva caminho: {LOCAL_MUSIC_PATH}/{id}.mp4 → Verifica se arquivo existe → Suporta Range Requests (seeking em vídeo) → Headers: Accept-Ranges, Content-Range, Content-Type: video/mp4 → Streaming via fs.createReadStream()

6. BANCO DE DADOS — DRIZZLE ORM

O banco de dados é PostgreSQL, acessado via Drizzle ORM. Schemas em: `lib/db/src/schema/`

6.1 Tabela: musicas

Armazena o catálogo de músicas de karaokê.

```
Schema: id: integer (PK) → ID da música = ID do vídeo no Bunny Stream artista: text → Nome do artista/banda
musica: text → Título da música inicio: text? → Letra inicial (para busca) hasVideo: boolean → Se vídeo está
disponível ( Bunny ou local ) createdAt: timestamp → Data de cadastro updatedAt: timestamp → Última
atualização
```

6.2 Tabela: sessions

Armazena sessões ativas de karaokê.

```
Schema: id: text (PK, 6 chars) → ID alfanumérico único (ex: L5662K) name: text → Nome descritivo mode: text
→ 'home' | 'party' queue: jsonb → Array de QueueEntry (fila de músicas) currentSongId: text? → ID da música
atual currentSingerName: text? → Nome do cantor atual currentSongAddedBy: text? → Device ID que adicionou
currentSongStartedAt: text? → ISO timestamp swapRequest: jsonb? → Dados de solicitação de troca updatedAt:
timestamp → Última atualização
```

6.3 Tabela: karaoke_users

Armazena assinantes do sistema.

```
Schema: id: serial (PK) → ID auto-incremento email: text (unique) → Email do assinante passwordHash: text? →
Hash bcrypt da senha subscriptionStatus: text → 'active'|'cancelled'|'expired' accessGranted: boolean → Se
acesso está liberado accessGrantedAt: timestamp? → Quando liberou expiresAt: timestamp? → Quando expira
activeSessionToken: text? → Token de sessão ativa createdAt: timestamp → Data de criação
```

6.4 Tabela: access_codes

Armazena cupons de acesso temporário.

```
Schema: id: serial (PK) code: text (unique) → Código de 8 caracteres hex durationMinutes: integer → Duração
em minutos label: text? → Descrição/label opcional redeemedBy: text? → Email de quem resgatou redeemedAt:
timestamp? → Data de resgate expiresAt: timestamp? → Data de expiração createdBy: text? → Quem criou o
código createdAt: timestamp → Data de criação
```

6.5 Comandos de Banco

```
pnpm --filter @workspace/db run push → Aplica schema ao banco (dev only) pnpm --filter @workspace/db run
migrate → Executa migrations pendentes pnpm --filter @workspace/db run studio → Abre Drizzle Studio (UI web)
DATABASE_URL no .env: → postgres://user:pass@host:port/dbname → Gerenciado automaticamente pelo Replit
```

7. BIBLIOTECAS COMPARTILHADAS (lib/)

Pacotes reutilizáveis entre frontend e backend. Gerenciados via pnpm workspaces.

7.1 api-spec (OpenAPI + Orval)

Define os contratos da API em OpenAPI 3.0. Fonte da verdade para geração automática de código.

```
Arquivo principal: lib/api-spec/openapi.yaml Comando de geração: pnpm --filter @workspace/api-spec run codegen
Gera automaticamente: → lib/api-client-react/src/generated/api.ts (React Query hooks) →
lib/api-client-react/src/generated/api.schemas.ts (Tipos TypeScript) → lib/api-zod/src/generated/api.ts
(Schemas Zod) Hooks gerados (exemplos): useSearchMusicas(params) → Hook para buscar músicas useGetMusica(id)
→ Hook para carregar música específica useCreateMusica() → Hook para cadastrar música (mutation)
useSyncBunny() → Hook para sincronizar Bunny Stream NÃO EDITE ARQUIVOS GERADOS MANUALMENTE! Sempre edite o
openapi.yaml e execute o codegen.
```

7.2 db (Drizzle ORM)

Pacote de acesso ao banco de dados. Compartilhado entre backend e scripts.

```
Arquivos principais: lib/db/src/index.ts → Exporta db client e tabelas lib/db/src/schema/musicas.ts → Schema
da tabela musicas lib/db/src/schema/sessions.ts → Schema da tabela sessions lib/db/src/schema/users.ts →
Schema da tabela karaoke_users lib/db/src/schema/access-codes.ts → Schema da tabela access_codes
lib/db/src/schema/index.ts → Barrel export de todos os schemas Uso no backend: import { db, musicasTable }
from '@workspace/db'; await db.select().from(musicasTable).where(eq(musicasTable.id, 123));
```

7.3 scripts (import-musicas.ts)

Script utilitário para importar catálogo de músicas a partir de arquivo .ini.

```
Comando: pnpm --filter @workspace/scripts run import-musicas -- --file /caminho/Dados_CT.ini Funcionamento: 1.
Lê arquivo .ini (detecta encoding: UTF-8 ou Latin-1) 2. Parseia seções [MUSICA_001], [MUSICA_002], etc. 3.
Extrai: ID, Artista, Música, Início 4. Insere no banco com INSERT ... ON CONFLICT DO NOTHING 5. Evita
duplicatas automaticamente Formato esperado do .ini: [MUSICA_001] ID=20704 ARTISTA=Abba MUSICA=Take A Chance
On Me INICIO=If You Change Your Mind
```

8. DEPLOY E CONFIGURAÇÃO

O deploy é feito automaticamente pelo Replit via workflows configurados.

8.1 Workflows (Replit)

Arquivo: .replit (configuração de workflows) Workflows configurados: artifacts/api-server: API Server → Comando: pnpm --filter @workspace/api-server run dev → Porta: definida pela variável PORT → Rota: /api/* artifacts/karaoke: web → Comando: pnpm --filter @workspace/karaoke run dev → Porta: definida pela variável PORT → Rota: /* (catch-all, menos específica) artifacts/mockup-sandbox: Component Preview Server → Comando: pnpm --filter @workspace/mockup-sandbox run dev → Usado para preview de componentes no canvas

8.2 Variáveis de Ambiente Obrigatórias

DATABASE_URL → Connection string do PostgreSQL (fornecida pelo Replit) PORT → Porta do servidor (fornecida pelo Replit) BASE_PATH → Path base do artifact (fornecido pelo Replit) Variáveis Opcionais: VITE_BUNNY_LIBRARY_ID → ID da library Bunny Stream (frontend) BUNNY_API_KEY → API key do Bunny Stream (backend) BUNNY_LIBRARY_ID → ID da library Bunny Stream (backend) LOCAL_MUSIC_PATH → Caminho para pasta de vídeos locais no servidor SESSION_SECRET → Secret para geração de tokens

8.3 Comandos Úteis

pnpm run typecheck → Verifica tipos em todo o monorepo pnpm run build → Compila todos os pacotes pnpm --filter @workspace/api-spec run codegen → Regera hooks e schemas pnpm --filter @workspace/db run push → Aplica schema ao banco pnpm --filter @workspace/karaoke run typecheck → Verifica tipos do frontend pnpm --filter @workspace/api-server run typecheck → Verifica tipos do backend

9. PADRÕES DE ARQUITETURA E DECISÕES

9.1 Contrato-First API

Todas as APIs são definidas primeiro no OpenAPI (openapi.yaml) e depois o código é gerado automaticamente. Isso garante que frontend e backend estejam sempre sincronizados.

9.2 Zod para Validação

Todas as entradas de API são validadas com Zod schemas antes de tocar o banco. Isso previne SQL injection e garante tipos corretos.

9.3 React Query para Estado Server-Side

Todas as requisições ao backend usam React Query (TanStack Query) hooks gerados pelo Orval. Isso fornece cache automático, revalidação e estados de loading/error.

9.4 Context Isolation

Cada contexto React isola uma única responsabilidade: → SessionContext: sessões de karaokê → AuthContext: autenticação de assinantes → LocalMusicContext: vídeos locais → TemporaryAccessContext: acesso por cupom → SearchContext: estado de busca compartilhado

9.5 Permissões na Fila

A regra mais importante do sistema: → Cada item da fila tem 'addedBy' (device ID) → Só quem ADICIONOU pode EDITAR ou REMOVER → O backend valida essa regra (403 se não for o dono) → O frontend esconde botões de ação para não-donos → Isso evita que convidados removam músicas de outros

9.6 Modos Casa vs Festa

Dois modos de operação com regras diferentes: Modo Casa (home): → Fila ilimitada (máx 30) → Qualquer um adiciona quantas músicas quiser → Ideal para família/amigos pequenos (até 4 pessoas) Modo Festa (party): → 1 música por dispositivo na fila → Todos cantam na mesma rodada → Ideal para eventos grandes (acima de 5 pessoas) → Previne que uma pessoa monopolize a fila

10. COMPONENTES REUTILIZÁVEIS

Componentes compartilhados entre múltiplas páginas.

10.1 Layout (layout.tsx)

Wrapper visual para todas as páginas. Inclui header com logo, barra de busca, botões de tema e autenticação.

Funcionalidades: → Logo + nome 'Karaokê Bora Cantar' (clicável, volta pra home) → Barra de busca com glow amarelo (dark mode) → Toggle tema claro/escuro (sol/lua) → Badge de tempo restante (acesso temporário) → Botão login/logout → Link admin (se autenticado) Estado visual do acesso temporário: → > 10 min: badge verde/amarelo normal → <= 10 min: badge pulso âmbar (alerta) → 0 min: redireciona automaticamente para /planos

10.2 AddToQueueDialog (add-to-queue-dialog.tsx)

Modal bottom sheet para inserir nome do cantor antes de adicionar à fila.

Interface: item: QueueCandidate | null → Música selecionada onConfirm: (singerName) => void → Callback com nome do cantor onCancel: () => void → Callback de cancelamento Comportamento: → Abre como bottom sheet (desliza de baixo) → Input grande e touch-friendly → Auto-focus após 100ms → Scroll automático para centralizar input → Botão confirmar desabilitado se nome vazio → Cancela ao clicar fora (backdrop click)

10.3 ThemeProvider (theme-provider.tsx)

Gerencia tema claro/escuro do sistema. Persiste preferência no localStorage.

Configuração: defaultTheme: 'dark' storageKey: 'karaoke-ct-theme-canary' → Classes CSS: .dark ou .light no <html> → Tailwind usa variáveis CSS para cores → Transição suave entre temas

11. COMPONENTES shadcn/ui

O sistema usa o design system shadcn/ui com Tailwind CSS v4. Componentes em: `artifacts/karaoke/src/components/ui/`

Componentes principais usados: Button → Botões com variantes: default, ghost, outline, destructive Input → Campos de texto com ícones e glow Card → Containers visuais com bordas sutis Dialog → Modais e popups Table → Tabelas de dados (admin) Tabs → Abas navegáveis Badge → Badges de status (coloridos) Skeleton → Placeholders de carregamento Toast → Notificações temporárias Form → Validação de formulários com react-hook-form Select → Dropdowns selecionáveis Sistema de Elevation (brilho em hover): → hover-elevate: brilho suave ao passar mouse → active-elevate: brilho mais forte ao clicar → Z-index: pseudo-elemento com z-index 999 → Causou problema com botão amarelo da TV (texto ficava sob o overlay) → Solução: usar `<div>` com `inline styles` ao invés de `<Button>`

12. GUIA DE MANUTENÇÃO E ATUALIZAÇÕES

12.1 Adicionar Nova Página

1. Criar arquivo em: `artifacts/karaoke/src/pages/nome-da-pagina.tsx` 2. Exportar componente React como default 3. Adicionar rota em: `artifacts/karaoke/src/App.tsx` `<Route path='/nova-rota' component={NovaPagina} />` 4. Adicionar link no Layout ou menu relevante 5. Executar: `pnpm --filter @workspace/karaoke run typecheck` 6. Testar no preview antes de publicar

12.2 Adicionar Nova Rota de API

1. Editar arquivo de rotas existente (ex: `musicas.ts`) ou criar novo arquivo em: `artifacts/api-server/src/routes/` 2. Definir validação Zod no `lib/api-spec/openapi.yaml` 3. Executar: `pnpm --filter @workspace/api-spec run codegen` 4. Implementar handler no backend 5. Usar hook gerado no frontend 6. Typecheck: `pnpm run typecheck`

12.3 Adicionar Nova Tabela no Banco

1. Criar schema em: `lib/db/src/schema/nova-tabela.ts` 2. Exportar em: `lib/db/src/schema/index.ts` 3. Push schema: `pnpm --filter @workspace/db run push` 4. Usar tabela no backend: `import from '@workspace/db'` 5. Atualizar `openapi.yaml` se necessário 6. Regenerar codegen: `pnpm --filter @workspace/api-spec run codegen`

12.4 Configurar Bunny Stream

1. Criar conta em [bunny.net](#) 2. Criar Video Library e obter Library ID 3. Obter API Key nas configurações 4. Configurar no Replit: `→ VITE_BUNNY_LIBRARY_ID` (frontend, shared) `→ BUNNY_API_KEY` (backend, shared) `→ BUNNY_LIBRARY_ID` (backend, shared) 5. Fazer upload dos vídeos .mp4 numerados 6. ID do arquivo no Bunny DEVE ser igual ao ID na tabela musicas 7. Sincronizar: `POST /api/musicas/sync-bunny` (painel admin)

12.5 Troubleshooting Comum

PROBLEMA: 'Carregando música...' eterno → CAUSA: `VITE_BUNNY_LIBRARY_ID` não configurado → SOLUÇÃO: Adicionar variável no painel Secrets do Replit
PROBLEMA: Botões amarelos com texto invisível → CAUSA: Sistema de elevation do `shadcn` sobrescreve cor do texto → SOLUÇÃO: Usar `<div>` com `inline styles` em vez de `<Button>`
PROBLEMA: Busca não retorna resultados → CAUSA: `hasVideo=false` nas músicas → SOLUÇÃO: Executar `sync-bunny` ou importar catálogo .ini
PROBLEMA: Sessão não sincroniza entre dispositivos → CAUSA: Polling parou (erro de rede) → SOLUÇÃO: Recarregar página ou verificar se sessão existe no banco
PROBLEMA: Typecheck falha após edição → CAUSA: Tipo incompatível ou import quebrado → SOLUÇÃO: `pnpm run typecheck` para ver erro completo → Se codegen quebrou: `pnpm --filter @workspace/api-spec run codegen`

12.6 Convenções de Código

NOMEAR: → Funções: camelCase (`handleAddToQueue`, `generateScore`) → Componentes: PascalCase (`TVPage`, `ScoreOverlay`) → Hooks: `use` + PascalCase (`useSession`, `useDebounce`) → Arquivos: kebab-case (`session-context.tsx`, `use-scoring.ts`) → Constantes: UPPER_SNAKE_CASE (`STORAGE_KEY`, `DEVICE_KEY`) ORGANIZAR: → Imports: React → Libs → Contexts → Hooks → Components → Utils → Estados: agrupar `useState` relacionados → Efeitos: `useEffect` com `cleanup` sempre → Callbacks: `useCallback` para funções passadas como props ESTILOS: → Preferir Tailwind classes sobre inline styles → Inline styles apenas quando necessário (elevation system) → Cores do tema via CSS variables (`hsl()`) → Dark mode como padrão

13. RESUMO DAS CHAVES DE LOCALSTORAGE

karaoke-ct-session-id → ID da sessão atual (para reconectar) karaoke-ct-device-id → ID único do dispositivo (para permissões) karaoke-ct-auth-token → Token de autenticação (assinantes) karaoke-ct-temp-access → Dados de acesso temporário (cupom) karaoke-ct-scoring-enabled → Toggle de pontuação (true/false) karaoke-ct-theme-canary → Preferência de tema (dark/light/system)

14. GLOSSÁRIO DE TERMOS

Sessão (Session) → Sala virtual de karaokê com fila de músicas Fila (Queue) → Lista ordenada de músicas a serem cantadas Modo Casa (home) → Sem limites, fila livre Modo Festa (party) → 1 música por dispositivo na fila Device ID → Identificador único do navegador (UUID) AddedBy → Quem adicionou a música (proteção de permissões) Swap → Troca de posição entre músicas na fila Scoring/Pontuação → Sistema de notas aleatórias após cantar Bunny Stream → Serviço de hospedagem de vídeos Codegen → Geração automática de código a partir do OpenAPI Barrel Export → Arquivo index.ts que re-exporta tudo Zod → Biblioteca de validação de schemas Drizzle → ORM type-safe para PostgreSQL Shadcn/ui → Design system baseado em Tailwind Monorepo → Repositório com múltiplos pacotes pnpm Workspace → Sistema de workspaces do pnpm